CSE 211 — Theory of Computation

Non-Regular Languages and the Pumping Lemma

Lesson notes · Exam approach · TF practice with solutions · Quick notes (Hakim Sir's portion)

How to Read These Notes

EXAM FOCUS	Exam importance, where it appears in the TF collection
MEMORIZE	Rules, tables and formulas to memorize
TRAP	Common mistakes and exam traps
EXAM-READY ANSWER	Model wording you can write in the exam
NOTE	Extra explanation / clarification
Grey box, monospace	Code, worked tables, derivations and hand-drawn sketches
Centered grey box	Formula to remember / final numerical result
Purple heading / ✓ green	Practice questions / their solutions

Contents

Exam Map — What the TF Papers Ask — a pumping-lemma part in every paper

Lesson 15 — The Pumping Lemma: Statement, Proof, Logic — pigeonhole, the three conditions, quantifiers, the adversary game

Lesson 16 — Pumping-Lemma Proofs — the 5-step template on every standard and TF language

Lesson 17 — Closure Arguments, Faulty Proofs, Pumping Lengths — complement/intersection tricks, spotting wrong arguments, minimum p

Quick Notes — Pumping Lemma — revision bullets + key diagram

> **NOTE** Slide numbers refer to the PDF pages of Lec06 - Non Regulars.pdf (50 pages). The minimum pumping length in Lesson 17 and the factorial inequality were checked by a program (pl_checks.py). Practice provenance: Actual = taken from a TF paper; Adapted = TF question with changed details; Constructed = new question in TF style.

Exam Map — What the TF Papers Ask

Session	Pumping-lemma questions
2023-24	Q2(a) Why must |xy| ≤ p hold in the pumping lemma? Example language and automaton (15) · Q4(c) {a^(n!) | n ≥ 1} is not regular (13)
2022-23	Q2(b) L = a^(2+3k) ∪ b^(10+12k): can 3, 5, 9, 24 be pumping lengths? Minimum pumping length (15)
2021-22	Q1(d) BLP's pumping-lemma arguments: palindromes, equal 0s/1s, {0^2n} — acceptable? (9) · Q4(a) Why is the pumping lemma for regular languages useful only for negative proofs? (4)
2020 (Jan term)	Q1(b) Skye's opinions: {ww} with y = (00)^k; {n0 = 3n1} with y = 0001 — what is wrong? (12)
2019-20	Q1(a) Primes not regular: 'Given p, let w = a^p…' — acceptable? (5) · Q4(c) {a^2n b^3n a^n | n ≥ 0} is not regular (15)
2017-18	Q3(b) Skye's pumping-lemma opinions on {ww}, {n0 = 2n1}, {0^n1^n0^n} — what is wrong? (17)
2016-17	Q4(c) Strings of 1s of prime length are not regular (12)
2015-16	Q4(c) Not regular: {0^n1^n2^n} and {0^i1^j | i > j} (6)
2014-15	Q2(a) Is {1^(n²) | n ≥ 0} regular? (pumping lemma) (10) · Q4(b) State and prove the pumping lemma (idea first, then proof) (12)
2013-14	Q6(a) State and prove the pumping lemma for regular languages (7⅓) · Q6(b) Properly nested parentheses are not regular (pumping lemma) (8) · Q8(c) Counter-example: every subset of a regular language is regular (6)
2012-13	Q5(b) Prove {0^n | n is a perfect square} is not regular (5⅓) · Q6(b) Technique to test whether L is regular — state and prove it (12⅓) · Q8(c) Prove {0^n 1 0^n | n > 1} is not regular (6)
2011-12	Q6(a) Technique to test whether L is regular — state and prove it (13⅓) · Q6(b) Prove not regular: {www | w ∈ {0,1}*} and {1^(n²) | n ≥ 0} (10)
2010-11	Q5 State and prove the pumping lemma (10) · Q6 Is {0^n 1^m 0^n | m, n ≥ 0} regular? (pumping lemma) (10)
2009-10	Q5 Pumping lemma: strings of 1s of prime length — regular or not? (10)
2008-09	Q5(a) Prove the pumping lemma for regular languages (statement given) (10) · Q5(b) Prove {0^n 1 0^n | n ≥ 1} is not regular (8)
2007-08	Q3(a) Pumping lemma: strings of 1s of prime length are not regular (10)
2006-07	Q3(a) Pumping lemma: strings with equal numbers of 0s and 1s (any order) are not regular (10)

EXAM FOCUS Every paper has one: either 'prove L is not regular' (10–15 marks; primes 4 times, squares 3 times, 0^n10^n twice) or, since 2017-18, 'find the mistakes in this student's pumping argument' (Skye / BLP: 2017-18, Jan-2020, 2019-20, 2021-22) and 2022-23's 'which numbers can be pumping lengths'. 'State and prove the lemma' was asked 7 times before 2015. Learn the 5-step template (Lesson 15) and the list of classic mistakes (Lesson 17) — they answer all of these.

Lesson 15 — The Pumping Lemma: Statement, Proof, Logic

Slides 1–11 (Lec06)

1. Why Finite States Force Repetition (sl. 4–7)

To accept 0^n1^n a machine must remember n — but a DFA has a fixed number p of states. A DFA is a directed graph; reading a string traces a walk. On a string of length ≥ p it visits p + 1 states (counting the start), so by the **pigeonhole principle** some state repeats, i.e. the walk contains a cycle. Split s = xyz: x leads to the repeated state, y goes round the cycle, z leads on to acceptance. The DFA cannot count the loops, so it must also accept xz, xyyz, xyyyz, …

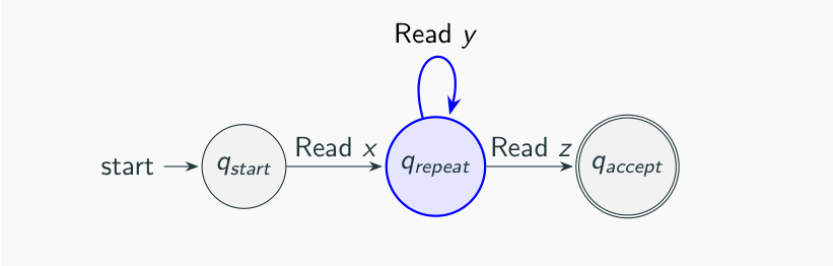


Figure 15.1 — A repeated state creates a cycle: x, y (loop), z (sl. 6)

2. The Lemma (sl. 8)

MEMORIZE If A is regular, there is a number p (the pumping length) such that every s ∈ A with |s| ≥ p can be split s = xyz with (1) xy^iz ∈ A for every i ≥ 0, (2) |y| > 0, (3) |xy| ≤ p.

3. Proof (textbook form of sl. 5–8)

EXAM-READY ANSWER Let M = (Q, Σ, δ, q1, F) be a DFA for A and p = |Q|. Take s = s1s2…sn ∈ A with n ≥ p and let r1, r2, …, r(n+1) be the states visited (r1 = q1, r(i+1) = δ(ri, si), r(n+1) ∈ F). Among the first p + 1 of them, r1 … r(p+1), two are equal by the pigeonhole principle (only p states): rj = rl with j < l ≤ p + 1. Put x = s1…s(j−1), y = sj…s(l−1), z = sl…sn. Then x takes M from r1 to rj, y from rj back to rl = rj, and z from rj to r(n+1) ∈ F. Hence M accepts xy^iz for every i ≥ 0 (loop y any number of times, including 0). |y| = l − j > 0 because j ≠ l, and |xy| = l − 1 ≤ p because l ≤ p + 1. ■

Why |y| > 0: the cycle consumes at least one symbol, otherwise nothing is pumped. **Why |xy| ≤ p:** the first repetition must happen within the first p steps (pigeonhole on r1 … r(p+1)) (sl. 8).

4. The Logic: Only Negative Proofs (sl. 9–10)

Lemma: ∃p ∀s (|s| ≥ p) ∃x,y,z (conditions) ∀i ≥ 0 : xy^iz ∈ A
To prove A NOT regular: ∀p ∃s ∀x,y,z (conditions) ∃i : xy^iz ∉ A

Turn	Who	What
1	Demon	chooses the pumping length p (unknown — keep it a variable)
2	You	choose s ∈ A with |s| ≥ p, written using p
3	Demon	splits s = xyz with |y| > 0, |xy| ≤ p — any way it likes
4	You	choose i ≥ 0 with xy^iz ∉ A — for EVERY possible split

EXAM-READY ANSWER Why only 'negative proofs' (2021-22 Q4(a)): the lemma says regular ⇒ pumpable. The converse is false — some non-regular languages satisfy all three conditions (e.g. {a^ib^jc^k | i = 0 or j = k}: pump the first symbol). So showing that strings can be pumped proves nothing; only a failure to pump proves non-regularity (modus tollens). To show a language IS regular, build a DFA/NFA/RE or use closure.

5. The 5-Step Template (sl. 11, 23)

1. Assume, for contradiction, that L is regular.
2. Let p be the pumping length (do not pick a value like p = 5).
3. Choose s ∈ L with |s| ≥ p, expressed in p; check both facts.
4. By the lemma s = xyz with |y| > 0, |xy| ≤ p. Use |xy| ≤ p to say what y must be (usually y = a^k, 1 ≤ k ≤ p).
5. Choose i (often 0 or 2) and prove xy^iz ∉ L for every such split. Contradiction ⇒ L is not regular.

Practice — Lesson 15

(a) Actual — 2013-14 Q6(a), 2014-15 Q4(b), 2010-11 Q5, 2008-09 Q5(a), 2011-12/2012-13 Q6. (b) Actual — 2023-24 Q2(a). (c) Actual — 2021-22 Q4(a).

(a) State and prove the pumping lemma for regular languages. First explain the idea, then the proof. (7–14)

(b) Justify why, in the pumping lemma, the decomposition w = xyz must satisfy |xy| ≤ p. Explain with an example language and its finite automaton. (15)

(c) Why is it said that the pumping lemma for regular languages is useful for 'negative proofs' only? (4)

✓ Solution

(a) Idea: §1 (finite states + long input ⇒ repeated state ⇒ pumpable loop). Statement: MEMORIZE box. Proof: EXAM-READY ANSWER in §3.

(b) Where it comes from: the repeated state is found among the first p + 1 states of the run, so the loop (and everything before it) lies inside the first p symbols. Why it matters: it lets the prover *locate* y. Example: A = {0^n1^n} is not regular. Suppose a DFA M with p states accepted it. Run M on 0^p1^p: the first p + 1 states are visited while reading only 0s, so a state repeats inside the 0-block; y = 0^k (k ≥ 1) and xz = 0^(p−k)1^p ∉ A. Without |xy| ≤ p the loop could lie anywhere — for example y could be 0^k1^k straddling the middle, and then pumping gives 0^p … strings out of order or, for other languages (e.g. {w | #0 = #1}), y = 01 pumps without leaving the language, and no contradiction could be forced. Automaton picture: for the DFA of 'contains 001' (Lec03 sl. 24) on 1001 the run is q, q, q0, q00, q001: the repeat (q, q) occurs within the first 4 = p steps, giving x = ε, y = 1, z = 001.

(c) EXAM-READY ANSWER in §4.

Lesson 16 — Pumping-Lemma Proofs

Slides 12–28 (Lec06)

1. The Slides' Five Examples (sl. 12–22)

Language	Choose s	Forced y	Pump	Why it fails
0^n1^n	0^p1^p	0^k	i = 0	p − k zeros, p ones
palindromes over {a,b}	a^pba^p	a^k	i = 0	left side p − k ≠ p
a^nb^nc^n	a^pb^pc^p	a^k	i = 2	p + k, p, p
w#w	0^p1#0^p1	0^k	i = 0	the two words differ
a^(n²)	a^(p²)	a^k, k ≤ p	i = 2	$p^2 < p^2 + k \le p^2 + p < (p+1)^2$

2. More Examples (sl. 26–27)

Language	Choose s	Pump	Contradiction
a^(n+1)b^n	a^(p+1)b^p	i = 0	no longer exactly one more a
a^mb^n, m > n	a^(p+1)b^p	i = 0	p + 1 − k ≤ p
a^nb^(2n)	a^pb^(2p)	i = 0	2p ≠ 2(p − k)
a^mb^nc^(m+n)	a^pb^pc^(2p)	i = 0	2p − k ≠ 2p
a^ib^jc^k, i = j or j = k	a^pb^pc^(p+1)	i = 0	p − r ≠ p and p ≠ p + 1
u#v, |u| = |v|	0^p#1^p	i = 0	lengths p − k, p
a^q, q prime	a^q, q > p prime	i = q + 1	length q(k + 1) composite
a^(2^n)	a^(2^m), 2^m > p	i = 2	2^m < 2^m + k < 2^(m+1)
a^mb^n, m = n or m = 2n	a^pb^p	i = 0	p − k is neither p nor 2p
a^mb^(m+n)c^n	a^pb^(2p)c^p	i = 0	2p − k ≠ 2p

TRAP Pumping a prime: choose a prime q > p (p itself may be composite!), and the pumped length is q + (i − 1)k, not (q + 1)k (sl. 34). The exponent i may depend on p and s (i = q + 1 is fine).

Exam Approach

1. Pick s so that the first p symbols are one block (then y is forced to be that symbol).
2. If the language counts, pump down (i = 0); if it needs special lengths (squares, primes, powers), pump up and show the new length falls strictly between two consecutive allowed values.
3. Write the final 'contradiction ⇒ not regular' sentence.

Practice — Lesson 16

(a) Actual — 2019-20 Q4(c). (b) Actual — 2023-24 Q4(c). (c) Actual — 2016-17 Q4(c), 2009-10 Q5, 2007-08 Q3(a). (d) Actual — 2014-15 Q2(a), 2011-12 Q6(b)(ii), 2012-13 Q5(b). (e) Actual — 2008-09 Q5(b), 2012-13 Q8(c), 2010-11 Q6. (f) Actual — 2011-12 Q6(b)(i). (g) Actual — 2013-14 Q6(b). (h) Actual — 2015-16 Q4(c).

(a) Prove {a^(2n) b^(3n) a^n | n ≥ 0} is not regular. (15)

(b) Prove {a^(n!) | n ≥ 1} is not regular. (13)

(c) Show that the strings of 1s whose length is prime do not form a regular language. (10–12)

(d) Is {1^(n²) | n ≥ 0} (equivalently {0^n | n a perfect square}) regular? (5–10)

(e) Show {0^n10^n | n ≥ 1} and {0^n1^m0^n | m, n ≥ 0} are not regular. (6–10)

(f) Prove {www | w ∈ {0,1}∗} is not regular.

(g) Σ = {(,)}: show the language of properly nested parentheses is not regular. (8)

(h) Prove {0^n1^n2^n | n ≥ 0} and {0^i1^j | i > j} are not regular. (6)

✓ Solution

(a) Assume L regular, p its pumping length. s = a^(2p)b^(3p)a^p ∈ L, |s| = 6p ≥ p. Since |xy| ≤ p and the first 2p symbols are a's, y = a^k, 1 ≤ k ≤ p. Pump down: xz = a^(2p−k)b^(3p)a^p. Membership would need 2p − k = 2n, 3p = 3n, p = n, i.e. n = p and k = 0 — impossible since k ≥ 1. So xz ∉ L; contradiction. L is not regular.

(b) Assume regular, p the pumping length. Let m = max(p, 2) and s = a^(m!) (|s| = m! ≥ m ≥ p). y = a^k with 1 ≤ k ≤ p ≤ m. Pump up (i = 2): |xy²z| = m! + k. Now m! < m! + k ≤ m! + m < m!·(m + 1) = (m + 1)! (because m! + m < m·m! + m! for m ≥ 2). So m! + k lies strictly between two consecutive factorials and is not a factorial: xy²z ∉ L. Contradiction. (Inequality checked by program for m ≤ 11.)

(c) Slides sl. 32–33: choose a prime q > p (infinitely many primes), s = 1^q. y = 1^k, 1 ≤ k ≤ p. With i = q + 1: |xy^(q+1)z| = q + qk = q(k + 1), a product of two integers ≥ 2 — composite. Contradiction.

(d) Not regular. s = 1^(p²); y = 1^k, 1 ≤ k ≤ p; i = 2 gives length p² + k with p² < p² + k ≤ p² + p < (p + 1)² = p² + 2p + 1 — not a square (sl. 20–21).

(e) s = 0^p10^p. |xy| ≤ p ⇒ y = 0^k inside the first block. i = 0: 0^(p−k)10^p — the blocks around the 1 differ ⇒ ∉ L. The same s (with m = 1) works for 0^n1^m0^n.

(f) s = 0^p1 0^p1 0^p1 ∈ L (w = 0^p1). y = 0^k in the first block. i = 0: 0^(p−k)1 0^p1 0^p1. A member www has three equal thirds, and since our string has three 1s each third must end in the 1s — the thirds would be 0^(p−k)1, 0^p1, 0^p1, unequal. Not in L.

(g) s = (^p)^p (balanced). y = (^k. i = 0: (^(p−k))^p has more) than (— not balanced. (Or via closure: L ∩ (∗)∗ = {(^n)^n} which is not regular.)

(h) 0^n1^n2^n: s = 0^p1^p2^p, y = 0^k, i = 2 gives p + k zeros (sl. 16–17 with a, b, c). {0^i1^j | i > j}: s = 0^(p+1)1^p, y = 0^k; pump DOWN (i = 0): 0^(p+1−k)1^p has p + 1 − k ≤ p zeros — not more than the 1s. (Pumping up would stay in L — choose i carefully.)

Lesson 17 — Closure Arguments, Faulty Proofs, Pumping Lengths

Slides 24–25, 29–50 (Lec06) + TF critique questions

1. Closure Proofs (sl. 24–25, 29–37)

When pumping is awkward, use closure properties to reduce to a known non-regular language:

- {0^m1^n | m ≠ n}: if regular, then 0∗1∗ − L = {0^n1^n} would be regular — contradiction (sl. 24).
- {w | #0(w) ≠ #1(w)}: complement is E = {#0 = #1}; E ∩ 0∗1∗ = {0^n1^n} — not regular (sl. 29–31). Careful: the complement of L is not 0^n1^n itself; the intersection with 0∗1∗ is what exposes it.
- Composite lengths: a∗ − (L_comp ∪ {ε, a}) = L_prime, which is not regular (sl. 35–37). Remember 0 and 1 are neither prime nor composite.
- Fibonacci lengths: s = a^(F_m) with F_(m−1) > p; i = 2 lands strictly between F_m and F_(m+1) (sl. 38–40).
- Equal numbers of ab and ba over {a, b, c}: intersect with (abc)∗(bac)∗ to get {(abc)^n(bac)^n}; pump with cases 3 ∤ |y| (length not divisible by 3) and 3 | |y| (block count drops) (sl. 41–47). Over {a, b} the same language IS regular — #ab − #ba depends only on the first and last symbols (sl. 48).

2. Classic Mistakes (sl. 23, 34, 40, 47) — the Checklist for Critique Questions

Mistake	Why it is wrong
Concluding 'L is regular' because some strings pump	the lemma is only a necessary condition
Choosing the split (x, y, z) yourself	the adversary chooses; you must handle every valid split
A y that violates |xy| ≤ p (e.g. y = 01 inside 0^(2p)1^p)	y must lie in the first p symbols
A fixed string (00011000) or one of length < p	s must be written in terms of p with |s| ≥ p
Choosing s = a^p for primes	p need not be prime, so s may not be in L
Claiming xy^iz ∉ L for ALL i	i = 1 gives s itself, which is in L — you need ONE bad i
'Proving' a regular language non-regular	if L is regular (e.g. {0^(2n)}) every proof must be flawed

3. Pumping Lengths

Any p that satisfies the lemma is 'a pumping length'; if p works, every larger p′ works too (|xy| ≤ p ≤ p′). The **minimum** pumping length is the smallest such p. Strings shorter than p need not be pumpable, so p must exceed the length of every member that cannot be pumped; for a regular language p = (number of DFA states) always works.

Exam Approach — 'What Is Wrong With This Argument?'

1. Is the conclusion even true? (Is L actually regular?)
2. Check s: in L? |s| ≥ p? written with p?
3. Check who chose the split, and whether y respects |xy| ≤ p and |y| > 0.
4. Check the quantifier on i and the direction of the conclusion (lemma only proves NON-regularity).
5. Give the correct proof (or the DFA if L is regular).

Practice — Lesson 17

(a) Actual — 2017-18 Q3(b). (b) Actual — Jan-2020 Q1(b). (c) Actual — 2019-20 Q1(a). (d) Actual — 2021-22 Q1(d). (e) Actual — 2022-23 Q2(b). (f) Actual — 2006-07 Q3(a). (g) Actual — 2013-14 Q8(c).

(a) Skye: (i) L1 = {ww | w ∈ {0,1}∗}: chooses 0^p0^p, x = ε, y = (00)^k, k ≤ p/2; since ∀i xy^iz ∈ L1, 'L1 is regular'. (ii) L2 = {w | n0(w) = 2n1(w)}: chooses 0^(2p)1^p, y = 01; since ∀i xy^iz ∈ L2 'we failed to demonstrate that L2 contradicts the pumping lemma'. (iii) L3 = {0^n1^n0^n}: chooses 0^k1^k0^k with k = ⌊p/3⌋; since ∀i xy^iz ∉ L3 for all possible y, L3 is not regular. What is wrong in each? (17)

(b) Jan-2020: (i) as (a)(i); (ii) L2 = {n0(w) = 3n1(w)}, string 0^(3p)1^p, y = 0001. (12)

(c) To show {a^n | n prime} is not regular we start: 'Given p, let w = a^p …'. Comment. (5)

(d) BLP: (i) palindromes — w = 00011000, 'xy lies in the leading 0s', xz not a palindrome ⇒ not regular; (ii) equal 0s and 1s — w = 0^(p/4)1^(p/4)0^(p/4)1^(p/4), 'y lies in the first 0s'; (iii) {0^(2n)} — w = 0^(2p), x = ε, y = 0, i = 0 gives odd length ⇒ not regular. Comment. (9)

(e) L = {a^(2+3k)} ∪ {b^(10+12k)}, k ≥ 0. Can 3, 5, 9, 24 be pumping lengths? Find the minimum pumping length. (15)

(f) Show that strings with equal numbers of 0s and 1s (any order) do not form a regular language. (10)

(g) Counter-example: every subset of a regular language is regular. (6)

✓ Solution

(a) (i) Wrong conclusion: pumping some strings never proves regularity, and Skye also chose x, y himself. In fact L1 is not regular: take 0^p1 0^p1; y = 0^k in the first block; i = 0 gives 0^(p−k)1 0^p1 ∉ L1. (ii) y = 01 is not a legal piece: the first p symbols of 0^(2p)1^p are all 0s, so |xy| ≤ p forces y = 0^k. Moreover he picked the split. Correct: i = 0 gives 0^(2p−k)1^p with n0 ≠ 2n1 ⇒ L2 not regular. (iii) The conclusion is right but the proof is not: |0^k1^k0^k| = 3⌊p/3⌋ may be less than p, so the lemma need not apply; with a short string y may cover several blocks; and 'xy^iz ∉ L3 for all i' is false for i = 1. Correct: s = 0^p1^p0^p, y = 0^k, i = 0.

(b) (i) As (a)(i). (ii) Same two errors: y = 0001 lies outside the first p symbols (all 0s), and the split is the adversary's. Correct proof: y = 0^k, i = 0 ⇒ n0 = 3p − k ≠ 3n1 ⇒ L2 is not regular.

(c) Not acceptable: p is arbitrary and may be composite, so a^p need not be in the language. Choose a prime q ≥ p (exists since primes are infinite) and w = a^q, then pump with i = q + 1 (Lesson 16 (c)).

(d) (i) Invalid as written: w = 00011000 is a fixed string, possibly shorter than p, and 'xy lies in the leading 0s' holds only if the leading block has length ≥ p. Repair: w = 0^p110^p; then y = 0^k and xz = 0^(p−k)110^p is not a palindrome — palindromes are indeed not regular. (ii) Invalid: the first block has only p/4 symbols, so y (within the first p symbols) may contain 1s — e.g. y = 0^(p/4)1^(p/4) keeps the counts equal under pumping. Repair: w = 0^p1^p. (iii) The conclusion is false — {0^(2n)} = (00)∗ is regular. The error: BLP chose y = 0; the adversary can choose y = 00, and then every xy^iz has even length.

(e) a-strings: a^m with m ≡ 2 (mod 3). Pumping a^j must keep length ≡ 2 (mod 3) and ≥ 2 when i = 0, so j must be a multiple of 3 and m − j ≥ 2: a^2 cannot be pumped; a^5, a^8, … can (j = 3). b-strings: b^m with m ≡ 10 (mod 12): j must be a multiple of 12 and m − j ≥ 10, so b^10 cannot be pumped and b^22, b^34, … need |y| = 12 ≤ p.

- p = 3, 5, 9: **no** — b^10 ∈ L has length ≥ p but cannot be pumped at all.
- p = 24: **yes** — every member of length ≥ 24 is a^m (m ≥ 26, take y = a^3) or b^m (m ≥ 34, take y = b^12); both fit in the first 24 symbols.
- Minimum: p must exceed 10 (b^10) and allow |y| = 12 for b^22, so p ≥ 12; p = 12 works (b^22: y = b^12 inside the first 12 symbols; all a-strings of length ≥ 12 pump with a^3). **Minimum pumping length = 12** (checked by program: 3, 5, 9, 11 fail; 12 and 24 work).

(f) Pumping: s = 0^p1^p, y = 0^k, i = 0 ⇒ unequal counts. Or closure: if E = {#0 = #1} were regular, then E ∩ 0∗1∗ = {0^n1^n} would be regular — contradiction.

(g) {0^n1^n} ⊆ Σ∗; Σ∗ is regular, {0^n1^n} is not.

Quick Notes — Pumping Lemma

Revision points for Lessons 15–17.

L15 · Statement and proof

- Regular ⇒ ∃p: every s ∈ L, |s| ≥ p, s = xyz, |y| > 0, |xy| ≤ p, xy^iz ∈ L ∀i ≥ 0.
- Proof: p = #states; p+1 states in first p steps ⇒ repeat ⇒ loop y.
- Only for non-regularity (necessary, not sufficient).
- Template: assume → p → s(p) → force y → choose i → contradiction.

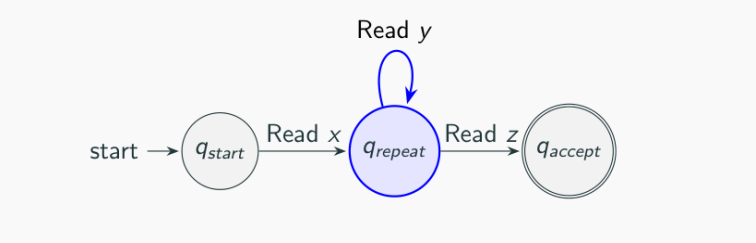
L16 · Standard proofs

- Counts: s = first block of length p, pump down.
- Squares: $p^2 < p^2 + k < (p+1)^2$. Primes: q > p prime, i = q+1 → q(k+1).
- n!: m = max(p,2), i = 2 → m! < m!+k < (m+1)!. 2^n: between 2^m and 2^(m+1).
- i > j: pump DOWN. Parentheses: (^p)^p.

L17 · Closure and critique

- L ∩ 0∗1∗ / 0∗1∗ − L exposes 0^n1^n. Composite = a∗ − (prime ∪ {ε, a}).
- Mistakes: wrong conclusion, choosing the split, |xy| > p, fixed short s, s = a^p for primes, 'for all i'.
- Pumping length: any p that works; larger p also work. a^(2+3k) ∪ b^(10+12k): minimum 12.

Key Diagrams to Reproduce



A repeated state creates a cycle (sl. 6)